

Time Complexity (Big-O)

CSC 240 Data Structures Fall 2025

Table of Contents

- [1. Objectives](#)
- [2. Introduction to Big-Oh](#)
- [3. History of O](#)
- [4. Big O for an array](#)
- [5. Linear time complexity: \$O\(n\)\$](#)
- [6. Drop constant terms](#)
- [7. Quadratic time complexity: \$O\(n^2\)\$](#)
- [8. Drop non-dominant terms](#)
- [9. Constant time complexity: \$O\(1\)\$](#)
- [10. Searching: \$O\(\log n\)\$](#)
- [11. Sorting: \$O\(n \log n\)\$](#)
- [12. PRACTICE Different terms for inputs](#)
- [13. Vectors - start and end](#)
- [14. Vectors - removing or inserting an element](#)
- [15. Big O: Wrap up](#)
- [16. Noweb chunk](#)
- [17. PRACTICE Timing your programs \[Bonus assignment\]](#)

1. Objectives

- [] Big-O: Time and space complexity
- [] History of Big-O
- [] Big-Omega, Big-O, Big-Theta
- [] Linear time complexity: $O(n)$
- [] Drop constant terms
- [] Quadratic time complexity: $O(n^2)$
- [] Constant time complexity: $O(1)$
- [] Searching: $O(\log n)$
- [] Sorting: $O(n \log n)$
- [] Vector complexity
- [] Big-O Overview

2. Introduction to Big-Oh

- Sometimes you need to compare two programs: Not by readability, conciseness, efficiency but by "complexity".
- **Measure 1:** Time complexity - not measured in time units but in the number of operations it takes to complete a task.
- **Measure 2:** Space complexity - memory taken up when it runs.
- This is measured mathematically (using a formula and convergence estimates).

3. History of O



- **Background:** The "O" in Big-O comes from the German word "Ordnung" (Engl. order) introduced by the German mathematician Paul Bachmann (1892). It refers to the order of growth of any function.
- "Big-O" is a general asymptotic upper bound notation, e.g.
 - Time complexity = running time as input grows
 - Space complexity = memory usage as input grows
 - I/O complexity = number of disk accesses or cache misses
 - Operation complexity = number of additions, multiplications etc.
 - Circuit complexity = Number of gates/depth of logic circuits
- Applies to any resource cost function $f(n)$ with input size n .

4. Big O for an array

- Given an array (1 2 3 4 5 6 7), and our job is to look for a number: Our worst case is looking for number 7, our best case is looking for 1, and our average case is looking for 4.
- Why is that?

These estimates only hold for a linear search - linear in the number of elements of the array: we compare each new element with the number.

1. Best case: target is $n=1$ - 1 comparison
2. Worst case: target is the last element $n=7$ - n comparisons
3. Average case: target can be anywhere - expect $(1 + 2 + \dots + n)/n = (n+1)/2$ comparisons.

- Terminology:

1. Best case: Ω ("Omega")
2. Worst case: O ("Big-oh")
3. Average case: Θ ("Theta")

5. Linear time complexity: $O(n)$

- Linear time complexity $O(n)$ is the easiest one to understand.
- The function `printItems` prints a loop index `i` exactly `n` times - it's $O(n)$.

```
#include <iostream>
using namespace std;

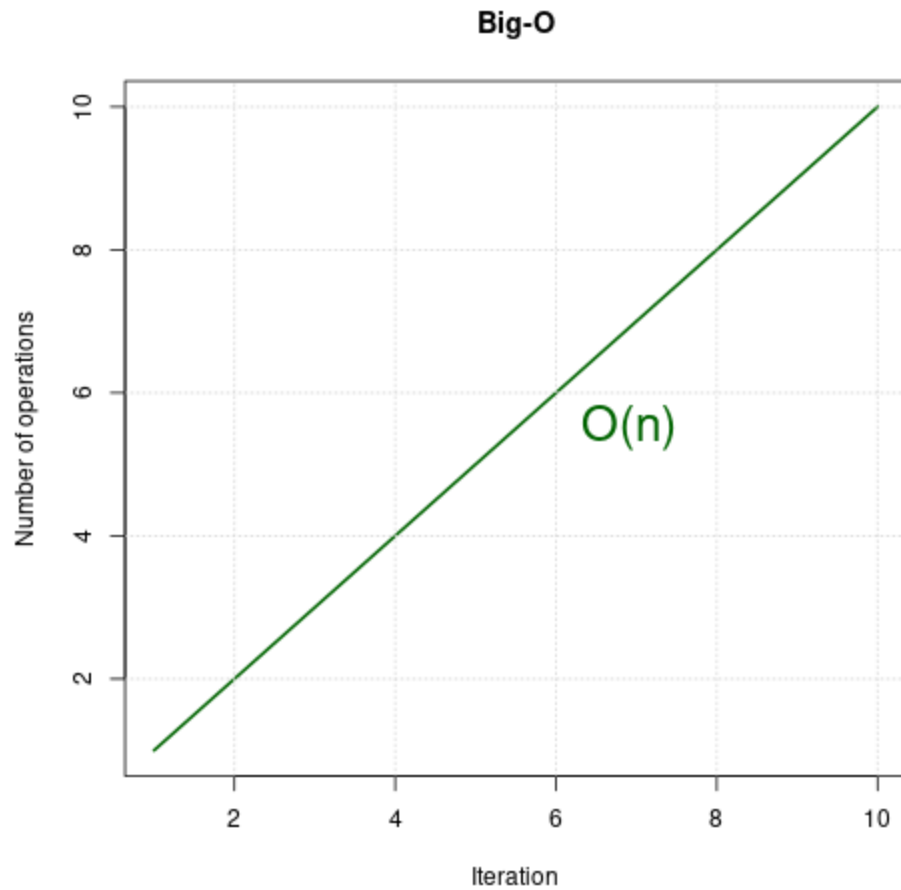
void printItems(int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << (i + 1) << " ";
    }
}

int main()
{
    printItems(10); // n = 10
    return 0;
}
```

1 2 3 4 5 6 7 8 9 10

- Let's look at this in a graph:

```
x <- 1:10
plot(x,x, type="l", col="darkgreen",lwd=2,
     ylab="Number of operations",
     xlab="Iteration")
grid()
text(x=7.5,y=5.5,labels=c("O(n)"),adj=1, col="darkgreen",cex=2)
title("Big-O")
```



6. Drop constant terms

- We're going to modify another for loop **serially**, after the first:

```
#include <iostream>
using namespace std;

void printItems(int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << (i + 1) << " ";
    }
    for (int j = n; j < n+n; j++)
    {
        cout << (j + 1) << " ";
    }
}

int main()
{
    printItems(10);
    return 0;
}
```

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

- In terms of complexity, $O(a * n) = O(n)$ because the effort is still growing linearly, at a rate of $(a * n + b)$.
- Visually, the curve for $O(n)$ and for $O(2n)$ are on top of one another (unless you have a constant term, which would only shift the curve up or down) so you won't be able to see a difference in growth rate.

7. Quadratic time complexity: $O(n^2)$

- Change the code to nest loops inside one another:

```
#include <iostream>
using namespace std;

void printItems(int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << i << j << endl;
        }
    }
}

int main()
{
    printItems(10);
    return 0;
}
```

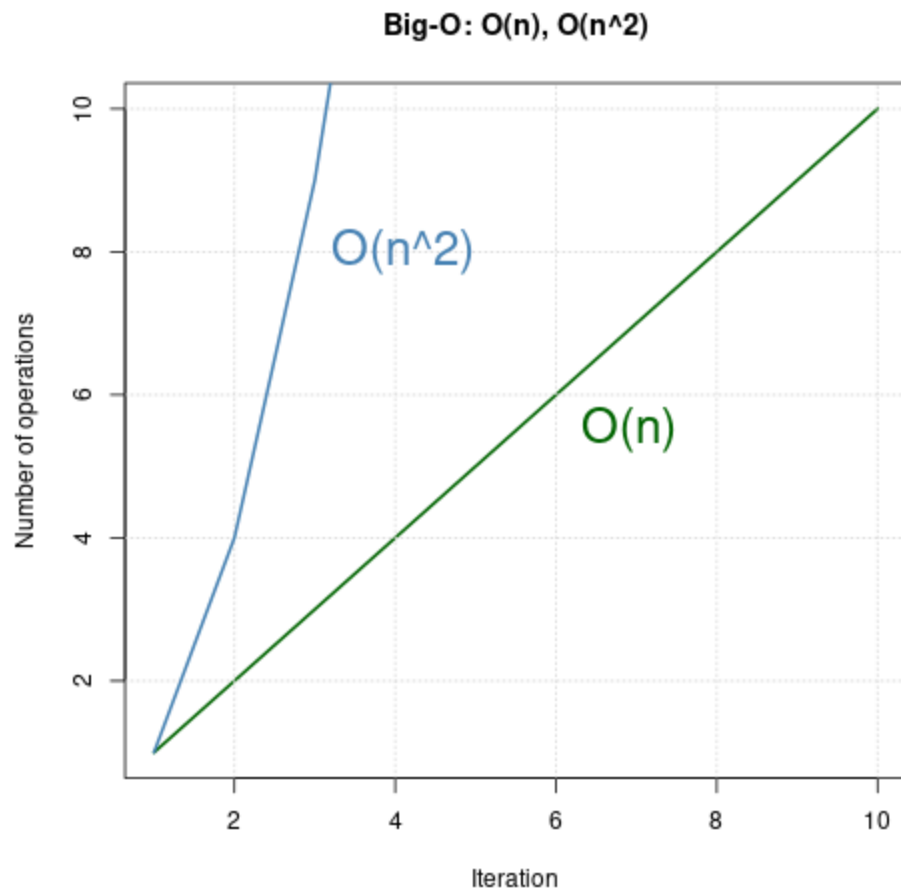
00
01
02
03
04
05
06
07
08
09
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27

28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92

```
93
94
95
96
97
98
99
```

- This loop has $10^2 = 100$ operations. It ran n^2 times. It is an order of magnitude less efficient than $O(n)$.
- Visually:

```
x <- 1:10
plot(x,x, type="l", col="darkgreen",lwd=2,
     ylab="Number of operations",
     xlab="Iteration")
text(x=7.5,y=5.5,labels=c("O(n)"),adj=1, col="darkgreen",cex=2)
title("Big-O: O(n), O(n^2)")
par(new=TRUE)
grid()
lines(x,x*x, type="l", col="steelblue",lwd=2,
     ylab="Number of operations",
     xlab="Iteration")
text(x=5,y=8,labels=c("O(n^2)"),adj=1, col="steelblue",cex=2)
```



8. Drop non-dominant terms

- Here we've added an additional for loop after the nested loops:

```
#include <iostream>
using namespace std;

void printItems(int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << i << j << endl;
        }
    }
    for (int k = 0; k < n; k++)
    {
        cout << k << endl;
    }
}

int main()
{
    printItems(10);
    return 0;
}
```

```
00
01
02
03
04
05
06
07
08
09
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
```

33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97

```
98
99
0
1
2
3
4
5
6
7
8
9
```

- This corresponds to $O(n^2) + O(n) = O(n^2 + n)$. Here, n^2 is the dominant, and n is the non-dominant term. When $n = 1000$, the dominant term is 10^6 and the non-dominant term is 10^3 only.
- **Dropping non-dominants** means that we drop all orders of n except the leading one.

9. Constant time complexity: $O(1)$

- For this algorithm, the number of operations does not change as n becomes larger - it's always 1 operation (the addition):

```
#include <iostream>
using namespace std;

int addItems(int n) {
    return n + n;
}

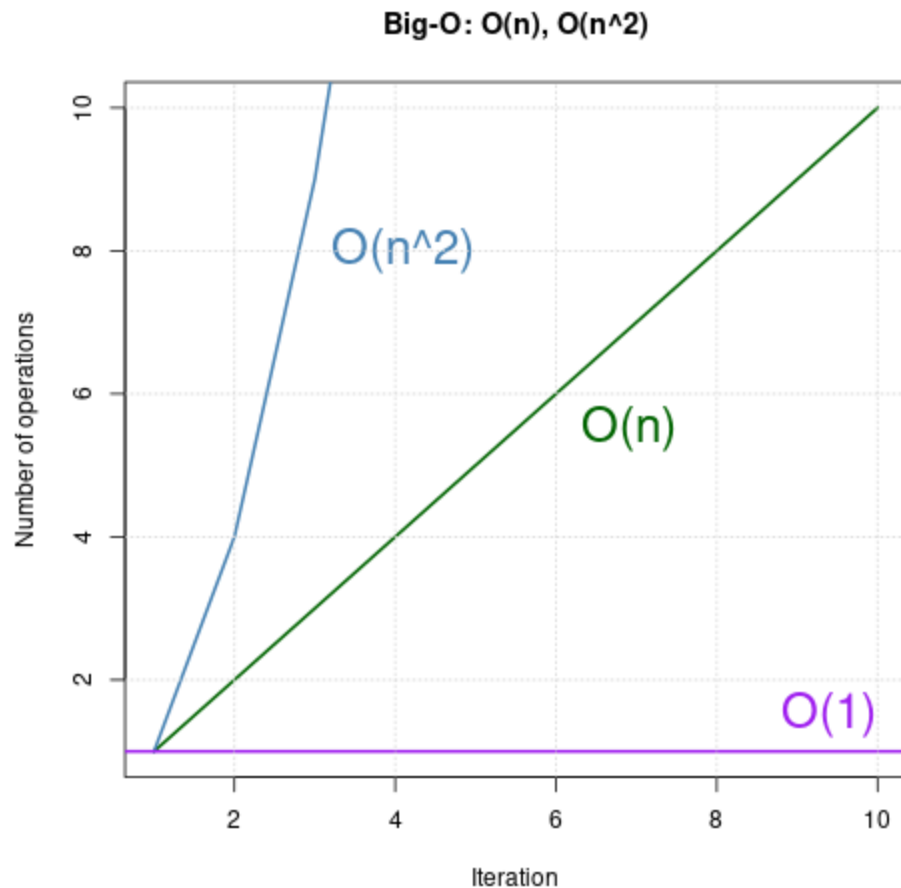
int main()
{
    cout << addItems(10) << endl;
    cout << addItems(100) << endl;
    return 0;
}
```

```
20
200
```

- What if we had $n + n + n$ instead? That's two operations, so we get $O(2)$, which we can simplify to $O(1)$.
- Extend the visualization:

```
x <- 1:10
plot(x,x, type="l", col="darkgreen",lwd=2,
     ylab="Number of operations",
     xlab="Iteration")
text(x=7.5,y=5.5,labels=c("O(n)"),adj=1, col="darkgreen",cex=2)
title("Big-O: O(n), O(n^2)")
par(new=TRUE)
lines(x,x*x, type="l", col="steelblue",lwd=2,
     ylab="Number of operations",
     xlab="Iteration")
text(x=5,y=8,labels=c("O(n^2)"),adj=1, col="steelblue",cex=2)
par(new=TRUE)
abline(h=1, lwd=2, col="purple")
```

```
text(x=10,y=1.5,labels=c("O(1)"),adj=1, col="purple",cex=2)
grid()
```



- $O(1)$ is the most efficient algorithm possible.

10. Searching: $O(\log n)$

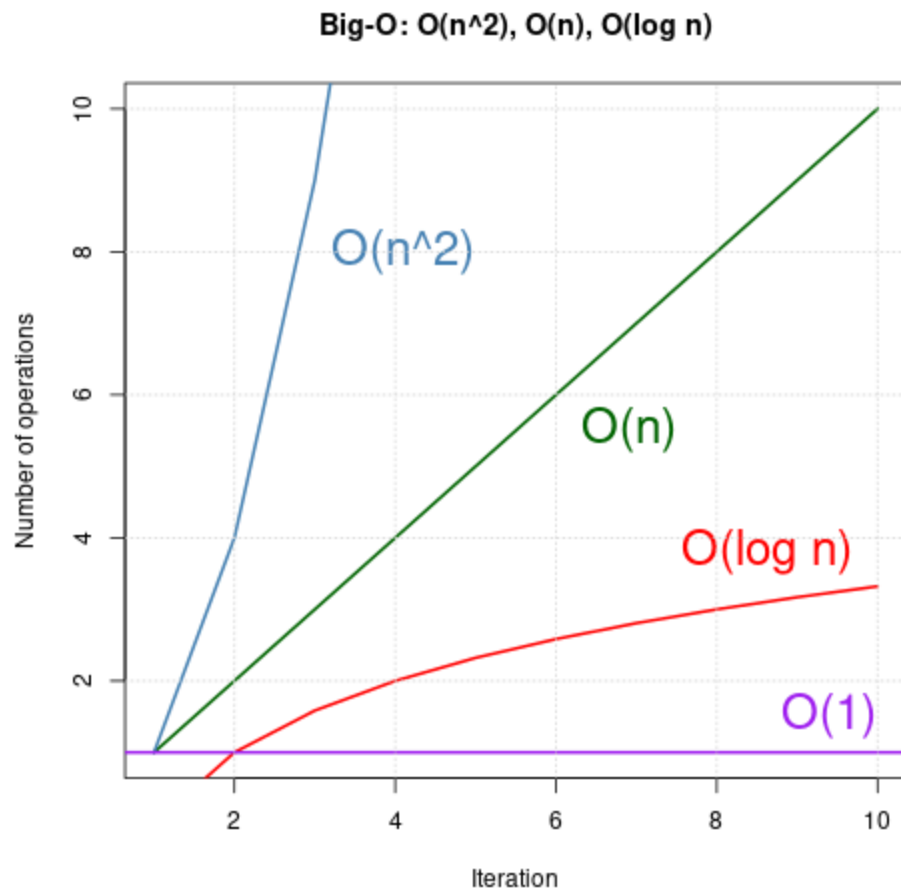
- To find any number in a **sorted** list of **eight** numbers using **binary search** (splitting the data set into consecutive halves until the number is found), takes **three** steps: since $2^3 = 8$, $\log_2(8) = 3$.
- Why?

Binary search on 8 elements:

- After 1 step: $8 \rightarrow 4$ elements left
- After 2 steps: $4 \rightarrow 2$ elements left
- After 3 steps: $2 \rightarrow 1$ element left
- For very large numbers: $\log_2(1,073,741,824) = 30$. To cut this number in half to get to down to one item requires 31 iterations (30 plus one final step which we can drop).
- So the binary search algorithm on a sorted array of n elements is $O(\log n)$ - which really is $O(\log_2 n)$.

- On the graph, $O(\log n)$ is very close to $O(1)$.

```
x <- 1:10
plot(x,x, type="l", col="darkgreen",lwd=2, # O(n)
      ylab="Number of operations", xlab="Iteration")
text(x=7.5,y=5.5,labels=c("O(n)"),adj=1, col="darkgreen",cex=2)
title("Big-O: O(n^2), O(n), O(log n)")
par(new=TRUE)
lines(x,x*x, type="l", col="steelblue",lwd=2, # O(n^2)
      ylab="", xlab="")
text(x=5,y=8,labels=c("O(n^2)"),adj=1, col="steelblue",cex=2)
par(new=TRUE)
lines(x,log2(x), type="l", col="red",lwd=2, # O(log n)
      ylab="",xlab="")
text(x=9.7,y=3.8,labels=c("O(log n)"),adj=1, col="red",cex=2)
par(new=TRUE)
abline(h=1, lwd=2, col="purple") # O(1)
text(x=10,y=1.5,labels=c("O(1)"),adj=1, col="purple",cex=2)
grid()
```



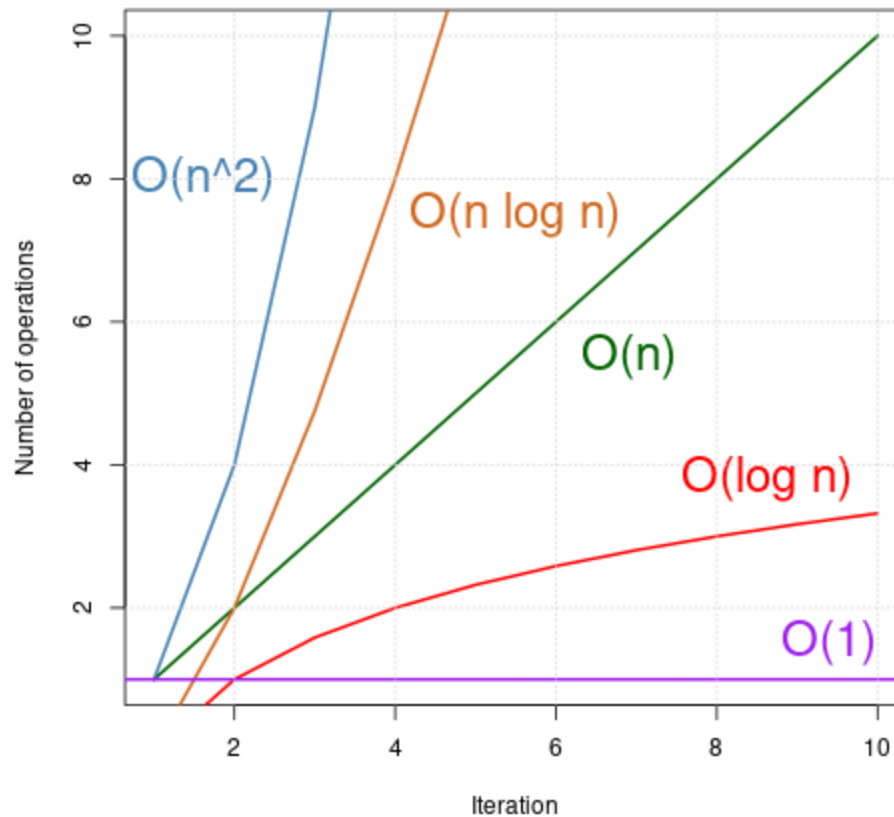
11. Sorting: $O(n \log n)$

- The most efficient that you can make a **sorting** algorithm like merge sort or quick sort is $O(n \log n)$.

- Merge sort: split in halves, sort each half, merge.
- Quick sort: pick a pivot, partition, sort recursively.
- Heuristic argument:
 - Comparing unordered items needs $n!$ possible orderings.
 - Stirling's approximation yields $\log_2(n!) \approx n \log_2 n - n$.
 - Lower bound after dropping linear term: $O(n \log n)$.
- Visualized, $O(n \log n)$ is between $O(n)$ and $O(n^2)$:

```
x <- 1:10
plot(x,x, type="l", col="darkgreen",lwd=2, # 0(n)
      ylab="Number of operations", xlab="Iteration")
text(x=7.5,y=5.5,labels=c("0(n)"),adj=1, col="darkgreen",cex=2)
title("Big-O: 0(n^2), 0(n log n), 0(n), 0(log n), 0(1)")
par(new=TRUE)
lines(x,x*x, type="l", col="steelblue",lwd=2, # 0(n^2)
      ylab="", xlab="")
text(x=2.5,y=8,labels=c("0(n^2)"),adj=1, col="steelblue",cex=2)
par(new=TRUE)
lines(x,x * log2(x), type="l", col="chocolate",lwd=2, # 0(n log n)
      ylab="", xlab="")
text(x=6.8,y=7.5,labels=c("0(n log n)"),adj=1, col="chocolate",cex=2)
par(new=TRUE)
lines(x,log2(x), type="l", col="red",lwd=2, # 0(log n)
      ylab="",xlab="")
text(x=9.7,y=3.8,labels=c("0(log n)"),adj=1, col="red",cex=2)
par(new=TRUE)
abline(h=1, lwd=2, col="purple") # 0(1)
text(x=10,y=1.5,labels=c("0(1)"),adj=1, col="purple",cex=2)
grid()
```

Big-O: $O(n^2)$, $O(n \log n)$, $O(n)$, $O(\log n)$, $O(1)$



12. PRACTICE Different terms for inputs

- **Task:** Write a function `printItems(int a, int b)` that runs two for loops in sequence:
 1. The first loop should print numbers from 1 to a.
 2. The second loop should print numbers from 1 to b.
 3. Call the function in main with `a=10` and `b=10`.
 4. Run the program and confirm the output.
 5. From the result: What is the runtime complexity?
- **Solution:**

```
#include <iostream>
using namespace std;

void printItems(int a, int b)
{
    for (int i = 0; i < a; i++)
    {
        cout << (i + 1) << " ";
    }
    for (int j = 0; j < b; j++)
    {
        cout << (j + 1) << " ";
    }
}
```

```

}

int main()
{
    printItems(10,10);
    return 0;
}

```

```

1 2 3 4 5 6 7 8 9 10 1 2 3 4 5 6 7 8 9 10

```

Discussion:

- Looks as if this was $O(n) + O(n) = O(2n) \rightarrow O(n)$.
- But really, this is $O(a) + O(b) = O(a + b)$.
- Same for a nested loop with two variables. This is not $O(n^2)$ but it is at best $O(a * b)$.

13. Vectors - start and end

- Most of what's true for vectors also applies to arrays.
- To add an item to the end of a vector, or when remove an item from its end, you don't have to touch any of the existing items:

```

vector<int> myVect {11,3,23,7};
cout << endl; for (int item : myVect) cout << item << " ";
myVect.push_back(17); // add 17 at end
cout << endl; for (int item : myVect) cout << item << " ";
myVect.pop_back();    // remove last item
cout << endl; for (int item : myVect) cout << item << " ";

```

```

11 3 23 7
11 3 23 7 17
11 3 23 7

```

- What's the complexity of vector operations `push_back` and `pop_back`?

vector operations `push_back` and `pop_back` are $O(1)$ because the function can jump in 1 step to the start/end of the vector.

14. Vectors - removing or inserting an element

- The first item of a vector can be retrieved with `begin()`, which is an iterator to the first element of `*this` ([doc](#)).

```

vector<int> myVect {11,3,23,7};
cout << endl; for (int item : myVect) cout << item << " ";
cout << "\nFirst item: " << *myVect.begin();

```

```

11 3 23 7

```

```
First item: 11
```

- We can use `begin()` to erase the first element - the method `erase` takes an iterator as argument (so we don't need the asterisk):

```
vector<int> myVect {11,3,23,7};  
cout << endl; for (int item : myVect) cout << item << " ";  
myVect.erase(myVect.begin());  
cout << endl; for (int item : myVect) cout << item << " ";
```

```
11 3 23 7  
3 23 7
```

- The problem is that now the new first element carries the index 1 instead of 0, so the index needs to be shifted down for every one of the $n-1$ remaining items, which is $O(n)$.
- The same occurs when we insert that item back at the start of the vector:

```
vector<int> myVect {11,3,23,7};  
cout << endl; for (int item : myVect) cout << item << " ";  
myVect.erase(myVect.begin()); // remove first item (value 11)  
cout << endl; for (int item : myVect) cout << item << " ";  
myVect.insert(myVect.begin(), 11); // re-insert 11 at start  
cout << endl; for (int item : myVect) cout << item << " ";
```

```
11 3 23 7  
3 23 7  
11 3 23 7
```

- **Summary:**

Adding or removing operation at the start is therefore $O(n)$ because of the reindexing while adding or removing from the end is $O(1)$.

- What if we insert an item somewhere not at the start or at the end?

```
vector<int> myVect {11,3,23,7};  
cout << endl; for (int item : myVect) cout << item << " ";  
myVect.insert(myVect.begin()+1, 99); // insert 99 between 1st and 2nd item  
cout << endl; for (int item : myVect) cout << item << " ";
```

```
11 3 23 7  
11 99 3 23 7
```

- Same for removing said element again:

```
vector<int> myVect {11,3,23,7};  
cout << endl; for (int item : myVect) cout << item << " ";
```

```
myVect.insert(myVect.begin()+1, 99); // insert 99 between 1st and 2nd item
cout << endl; for (int item : myVect) cout << item << " ";
myVect.erase(myVect.begin()+1); // remove 99 from 2nd position
cout << endl; for (int item : myVect) cout << item << " ";
```

```
11 3 23 7
11 99 3 23 7
11 3 23 7
```

- Now, $n-2$ elements have to be reindexed - this is still $O(n-2) \rightarrow O(n)$.

- **Summary:**

Inserting or removing somewhere away from the beginning or end does not change the situation because reindexing always introduces only a constant to the Big O argument: $O(1/2 * n) \rightarrow O(n)$.

- How about **searching** or **accessing** a particular value: What if we want to find the value 7 in our (unsorted) vector? We start with the first item and move down the vector until we found it. That's $O(n)$.
- Looking up by **index** instead means that we can go directly to the index, so it is $O(1)$. In R, if we perform `which(x = 7)` to get the index of `x` that has the value 7, that's an $O(n)$ operation but once we've got that index, using it incurs only an $O(1)$ burden.

- **Summary:**

Operation	Description	Big O
<code>push_back</code>	Add item to end	$O(1)$
<code>pop_back</code>	Remove item from end	$O(1)$
<code>begin()</code>	Get iterator to first item	$O(1)$
<code>erase(begin())</code>	Remove item from start	$O(n)$
<code>insert(begin(), value)</code>	Insert at start	$O(n)$
<code>insert(mid-position, value)</code>	Insert in middle	$O(n)$
<code>erase(mid-position)</code>	Remove from middle	$O(n)$
value lookup (by value)	Linear search	$O(n)$
access by index	Random access	$O(1)$

15. Big O: Wrap up

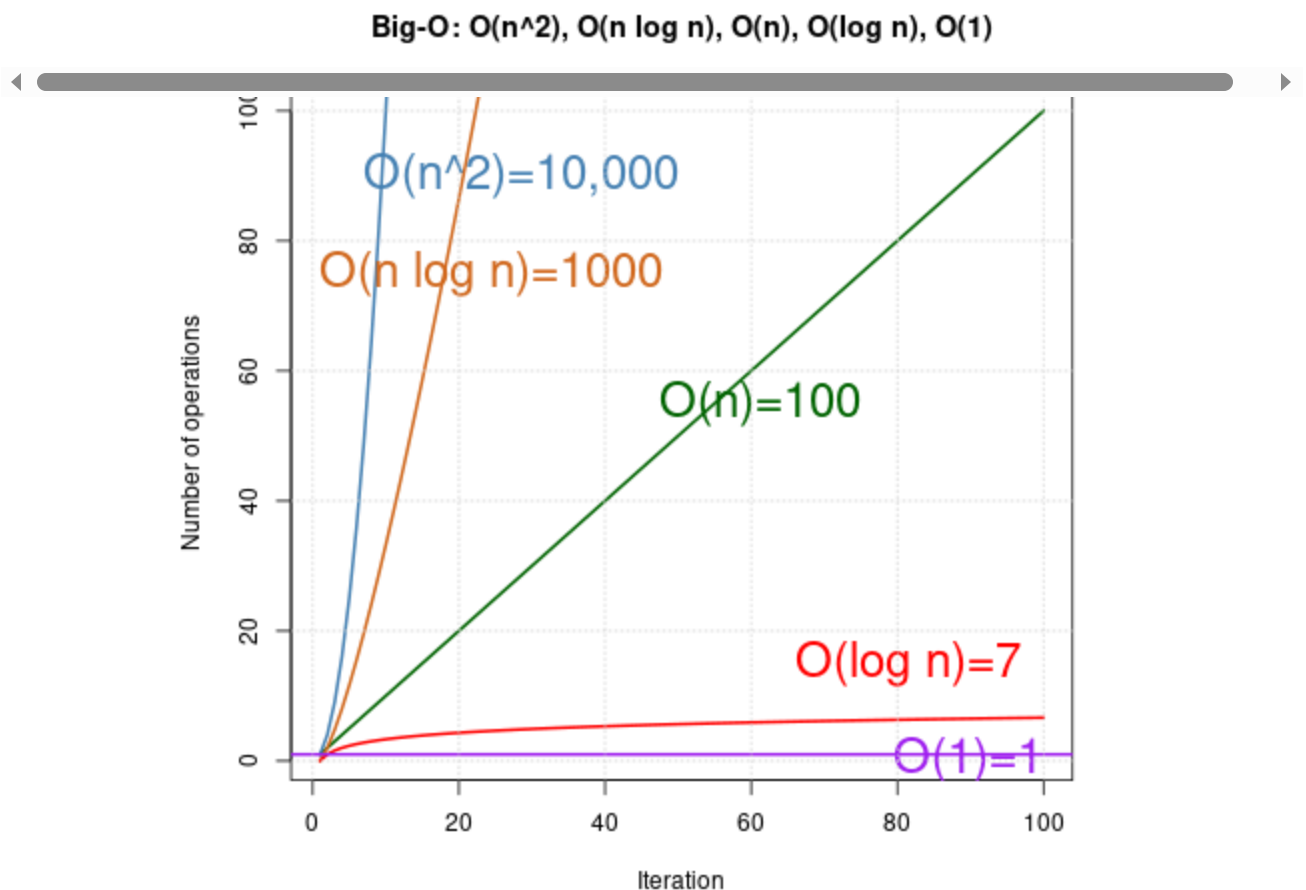
- Visualize the different Big O's for $n = 100$:

```
x <- 1:100
plot(x,x, type="l", col="darkgreen",lwd=2, # O(n)
     ylab="Number of operations", xlab="Iteration")
text(x=7.5*10,y=5.5*10,labels=c("O(n)=100"),adj=1, col="darkgreen",cex=2)
title("Big-O: O(n^2), O(n log n), O(n), O(log n), O(1)")
par(new=TRUE)
```

```

lines(x,x*x, type="l", col="steelblue",lwd=2, #  $O(n^2)$ 
      ylab="", xlab="")
text(x=50,y=9*10,labels=c(" $O(n^2)=10,000$ "),adj=1, col="steelblue",cex=2)
par(new=TRUE)
lines(x,x * log2(x), type="l", col="chocolate",lwd=2, #  $O(n \log n)$ 
      ylab="", xlab="")
text(x=6.8*10-20,y=7.5*10,labels=c(" $O(n \log n)=1000$ "),adj=1, col="chocolate",cex=2)
par(new=TRUE)
lines(x,log2(x), type="l", col="red",lwd=2, #  $O(\log n)$ 
      ylab="",xlab="")
text(x=97,y=15,labels=c(" $O(\log n)=7$ "),adj=1, col="red",cex=2)
par(new=TRUE)
abline(h=1, lwd=2, col="purple") #  $O(1)$ 
text(x=100,y=0,labels=c(" $O(1)=1$ "),adj=1, col="purple",cex=2)
grid()

```



- Visualize it for $n=1000$:

```

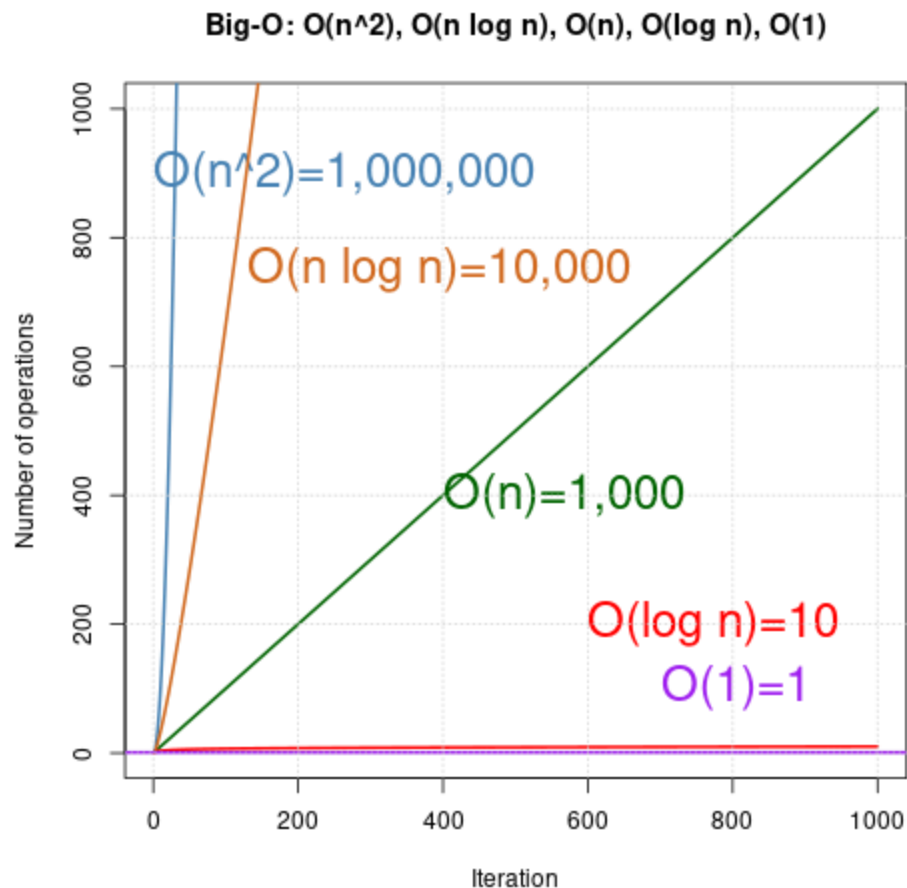
x <- 1:1000
plot(x,x, type="l", col="darkgreen",lwd=2, #  $O(n)$ 
     ylab="Number of operations", xlab="Iteration")
text(x=400,y=400,labels=c(" $O(n)=1,000$ "),adj=0, col="darkgreen",cex=2)
title("Big-O:  $O(n^2)$ ,  $O(n \log n)$ ,  $O(n)$ ,  $O(\log n)$ ,  $O(1)$ ")
par(new=TRUE)
lines(x,x*x, type="l", col="steelblue",lwd=2, #  $O(n^2)$ 

```

```

    ylab="", xlab="")
text(x=0,y=900,labels=c("O(n^2)=1,000,000"),adj=0, col="steelblue",cex=2)
par(new=TRUE)
lines(x,x * log2(x), type="l", col="chocolate",lwd=2, # O(n log n)
      ylab="", xlab="")
text(x=150-20,y=750,labels=c("O(n log n)=10,000"),adj=0, col="chocolate",cex=2)
par(new=TRUE)
lines(x,log2(x), type="l", col="red",lwd=2, # O(log n)
      ylab="",xlab="")
text(x=600,y=200,labels=c("O(log n)=10"),adj=0, col="red",cex=2)
par(new=TRUE)
abline(h=1, lwd=2, col="purple") # O(1)
text(x=700,y=100,labels=c("O(1)=1"),adj=0, col="purple",cex=2)
grid()

```



- **Terminology:**

$O(n^2)$	Loop within a loop
$O(n)$	Proportional
$O(\log n)$	Divide and conquer
$O(1)$	Constant

- A few overview graphs:

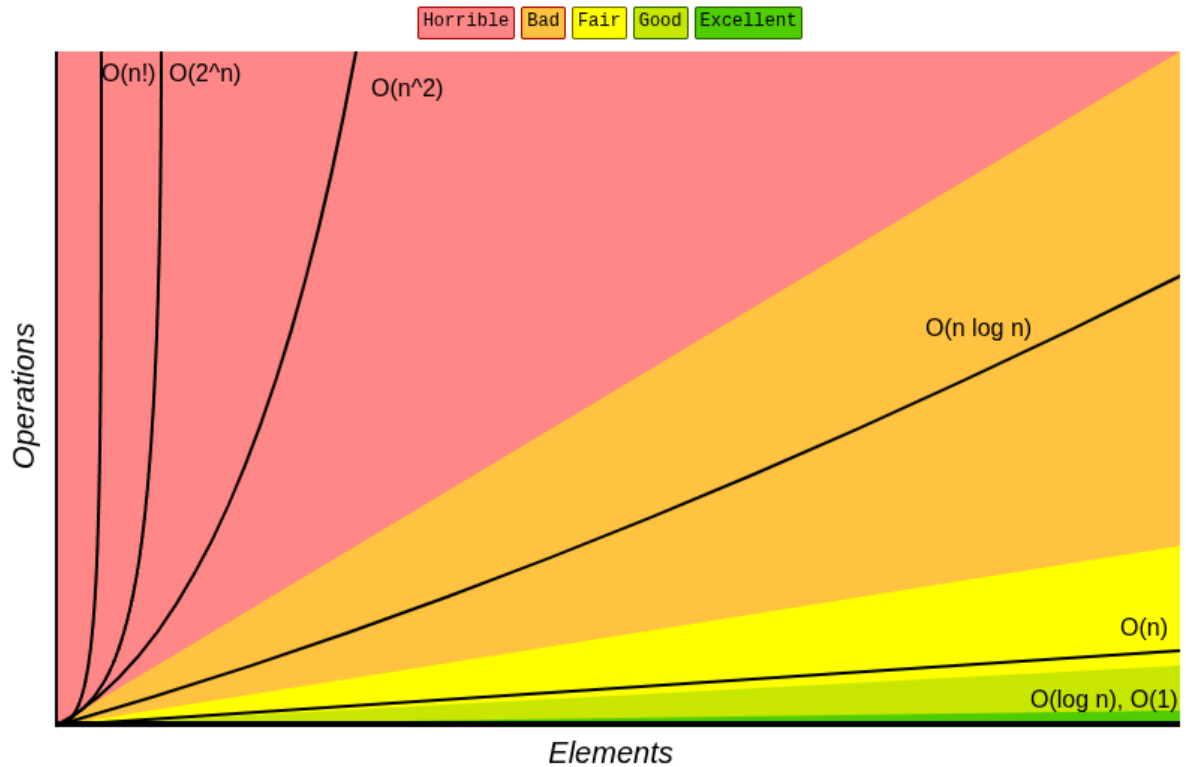


Figure 1: Operations as a function of number of elements (Source: BigOcheatsheet.com)

- In this graph, the efficiency of different search, access, delete and insert operations on a variety of data structures is shown: Only the array is a basic data structure, all the other ones have to be built. Both time and space complexity are indicated. Notice that all but one data structure has the same space complexity.

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
<u>Array</u>	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Stack</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Queue</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Singly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Doubly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Skip List</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n \log(n))$
<u>Hash Table</u>	N/A	$\theta(1)$	$\theta(1)$	$\theta(1)$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Binary Search Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Cartesian Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>B-Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Red-Black Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Splay Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>AVL Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>KD Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$

Figure 2: Common data structure operations (Source: BigOcheatsheet.com)

- Here is an overview of the (worst case) efficiency of different sorting algorithms on arrays, both in terms of time and space. Here, both time and space are important to consider.

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
<u>Mergesort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Timsort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Heapsort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(1)$
<u>Bubble Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Insertion Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Selection Sort</u>	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Tree Sort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(n)$
<u>Shell Sort</u>	$\Omega(n \log(n))$	$\Theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
<u>Bucket Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n^2)$	$O(n)$
<u>Radix Sort</u>	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$	$O(n+k)$
<u>Counting Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$
<u>Cubesort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$

Figure 3: Efficiency of array sorting algorithms (Source: BigOcheatsheet.com)

- For the array algorithms, if you have (almost) sorted data, some of the more primitive sorting algorithms can beat quicksort/mergesort.

16. Noweb chunk

- Noweb** chunk: print myVect

```
cout << endl; for (int item : myVect) cout << item << " ";
```

17. PRACTICE Timing your programs [Bonus assignment]

- To measure how fast your code is, you can wrap the loop call with <chrono> timers. Example:

```
#include <iostream>
#include <chrono>
```

```

using namespace std;
using namespace std::chrono;

void printItems(int n)
{
    for (int i = 0; i < n; i++)
        cout << (i + 1) << " ";
}

int main()
{
    auto start = high_resolution_clock::now();

    printItems(1000); // n = 10

    auto stop = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(stop - start);

    cout << "\nTime taken: " << duration.count() << " microseconds\n";
    return 0;
}

```

```

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 3
Time taken: 42 microseconds

```

- There are two problems here:
 1. Printing is time-expensive because interaction with I/O devices is very slow and time-consuming. Switch it off.
 2. The compiler does a lot of optimization. You can switch it off to some extent by declaring variables volatile.
- Improved example:
 1. No printing
 2. Loop in doWork is wrapped in a repetitive loop
 3. The measured time (for each repetition) is averaged.

```

#include <iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;

void doWork(int n)
{
    volatile int sink = 0; // prevent optimization
    for (int i = 0; i < n; i++)
        sink += i;
}

int main()
{
    int R = 100; // number of repetitions
    int n = 10000; // input size
    long long total = 0;

    for (int r = 0; r < R; r++) {
        auto start = high_resolution_clock::now();

        doWork(n);
    }
}

```

```

    auto stop = high_resolution_clock::now();
    total += duration_cast<nanoseconds>(stop - start).count();
}

cout << "Average time: " << (total / R) << " ns\n";
return 0;
}

```

Average time: 6015 ns

- **Task:** Test the three scenarios from the lecture using two serial loops, a nested loop, and a single operation, to complete the following table (the first entry comes from the code above).

Complexity	Type	n	R	Time
O(n)	Proportional	10000	100	3098 ns
O(n ²)	Loop within a loop	10000	100	
O(1)	Constant	10000	100	

- Submit the URL to your OneCompiler.com file and the table (copy and paste it into a text field, or submit an Emacs Org-mode file).

Solution

1. Loop within a loop - O(n²)

```

#include <iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;

void doWork(int n)
{
    volatile int sink = 0; // prevent optimization
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            sink += i + j;
}

int main()
{
    int R = 100; // number of repetitions
    int n = 10000; // input size
    long long total = 0;

    for (int r = 0; r < R; r++) {
        auto start = high_resolution_clock::now();

        doWork(n);

        auto stop = high_resolution_clock::now();
        total += duration_cast<nanoseconds>(stop - start).count();
    }

    cout << "Average time: " << (total / R) << " ns\n";
}

```

```
    return 0;
}
```

Average time: 37413924 ns

2. One operation - $O(1)$

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;

void doWork(int n)
{
    volatile int sink = 0; // prevent optimization
    sink += 1;
}

int main()
{
    int R = 100; // number of repetitions
    int n = 10000; // input size
    long long total = 0;

    for (int r = 0; r < R; r++) {
        auto start = high_resolution_clock::now();

        doWork(n);

        auto stop = high_resolution_clock::now();
        total += duration_cast<nanoseconds>(stop - start).count();
    }

    cout << "Average time: " << (total / R) << " ns\n";
    return 0;
}
```

Average time: 16 ns

Author: Marcus Birkenkrahe

Created: 2026-03-14 Sat 20:52